

## Parcial 2 Analisis de Algoritmos

Link al Repositorio

### Integrantes

- Mathew Cordero Aquino [22982]

### Instrucciones

- Debe realizar este examen de forma individual.
- Debe entregar la respuesta a las preguntas planteadas acompañada de un análisis riguroso y explicación pertinente, i.e., no se aceptarán respuestas sin procedimiento ni explicación y la nota será automáticamente de cero puntos.
- Entregue un PDF con la resolución para su parcial y además grabe un video de no más de 5 minutos de longitud en donde muestre la ejecución de los programas planteados en cada problema que usted realice. Súbalo a YouTube como no listado e inclúyalo en su PDF de respuestas.
- Duración: este examen se encontrará habilitado para su realización desde el [Fecha Inicial] hasta el [Fecha Final], por lo que podrá realizarlo a su conveniencia en los tiempos que mejor se ajusten a su itinerario. Solo podrá subir sus respuestas una vez, así que esté seguro de las mismas.
- Este parcial contiene múltiples problemas, con diferente valoración de puntos. Elija aquella combinación de problemas que lo lleve a un punteo completo. Si elige problemas extra, se tomarán como puntos extra que puede utilizar en otras entregas. Se hará una regla de tres con base en la puntuación total del parcial en puntos netos para definir cuántos puntos netos extra tendrá.
- Buena suerte y recuerde que Diosito lo/la está viendo

### Problemas

#### 2.1 Problema 1 [40 puntos]

Dado un numero positivo  $n$  y un teclado móvil de un Nokia 3230, que tiene dígitos del 0 al 9 asociados con cada tecla, se desea contar el total de combinaciones posibles de dígitos de longitud  $n$ . Podemos comenzar con cualquier dígito y presionar solo cuatro teclas adyacentes a cualquier dígito. El teclado también contiene las teclas  $*$  y  $\#$ , las cuales no están permitidas presionar. Por ejemplo para:

```
// Cantidad de digitos n
n = 2
```

```
// Mapping del teclado del Nokia
keypad = [
[ ' 1 ' , ' 2 ' , ' 3 ' ] ,
[ ' 4 ' , ' 5 ' , ' 6 ' ] ,
[ ' 7 ' , ' 8 ' , ' 9 ' ] ,
[ ' * ' , ' 0 ' , ' #' ]
]
```

Input : n = 2

Output : 36

Explicacion : El total de combinaciones posibles son 36 [ 0 0 , 0 8 , 1 1 , 1 2 , 1 4 , 2 1 , 2 2 , 2 3 , 2 5 , 3 2 , 3 3 , 3 6 , 4 1 , 4 4 , 4 5 , 4 7 , . . . , 9 6 , 9 8 , 9 9]

- Explique porque este problema exhibe subestructura optima.

Tiene subestructura optima porque el numero de secuencias se puede calcular del numero de secuencias - 1 .

Entonces definimos que la cantidad que el numero de secuencias es:

$$count(n, d) = \sum_{adj(d)} count(n-1, x)$$

Por lo tanto es optima porque

Para calcular las combinaciones de longitud  $n$  que terminan en el dígito  $d$ , solo necesitamos las combinaciones de longitud  $n-1$  que terminan en dígitos adyacentes a  $d$

- Explique una idea/solucion que exhiba subproblemas traslapados e indique como los mismos subproblemas se computan repetidamente

Bueno para hacer esto podemos definir un arbol de recursion de la siguiente manera

Podemos ver que recorriendo el arbol de recursion, en cada siguiente nivel se recorren los vecinos del nodo padre, y se encuentran el numero de combinaciones desde ese vecino hasta el padre.

Aqui es donde ocurren los problemas traslapados. De hecho podemos ver que se repite calculos por ejemplo en cada uno de los niveles se calcula  $count(2, 6)$  y  $count(2,2)$  que llegan siempre los mismos a  $count(1,1)$  se calcula cada vez que consideramos llegar a '2' o '4'

Para ello usaremos programacion dinamica con memorizacion guardando en una tabla lo ya calculado, asi cuando se repita la misma entrada o input podremos regresar a devolver el mismo output que nos dio la vez pasada. Esto en una tabla grid con los valores de  $n$  y  $d$ .

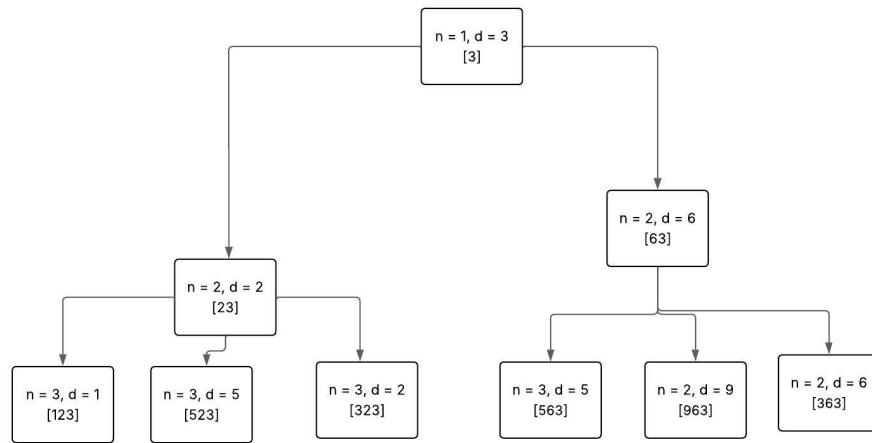


Figure 1: alt text

- Sabemos que los problemas con subestructura optima y subproblemas traslapados pueden resolverse utilizando programacion dinamica, donde las soluciones de subproblemas se memoizan en lugar de calcularse repetidamente. Escriba un codigo en Python con enfoque de memoizacion top-down que resuelva este problema. Coloque un enlace a un GitHub Gist privado con la solucion. Recuerde que debe crear tambien un video en donde muestre la ejecucion de su codigo.

Link del Codigo

Link al Video

- Encuentre el tiempo de complejidad para este algoritmo. Recuerde, deje su procedimiento.

Mi codigo es

```

def algoritmo_solucion(digit, length):
    if (digit, length) in valores_calculados:
        return valores_calculados[(digit, length)]

    if length == 1:
        return 1

    count = 0
    for siguiente_digito in vecinos[digit]:
        count += algoritmo_solucion(siguiente_digito, length - 1)

    valores_calculados[(digit, length)] = count
  
```

```

    return count

total_combinations = 0
for digit in '0123456789':
    total_combinations += algoritmo_solucion(digit, n)

return total_combinations

```

Para esto vamos a calcular la formula de recursion del algoritmo.

La formula de recursion es

$$T(S) = \begin{cases} 1, & \text{si } l = 1 \\ \sum_{d' \in \text{vecinos}(d)} f(d', l-1), & \text{si } l > 1 \end{cases}$$

Donde  $l$  es el lenght del parametro y el caso base el  $n$  que se pasa como parametro lenght en la funcion sera 1.

Y  $d'$  es el siguiente digito alcanzado apartir de  $d$ .

Para detectar la sumatoria tendríamos que

$$f(d, l) = \sum f(d', l-1)$$

Para cada par  $(d, l)$  haces tantas sumas como digitos.

El numero de vecinos por digito es  $k$  y  $k \leq 8$

Por ello  $f(d, l)$  realiza  $k$  llamadas recursivas a  $f(d, l-1)$  pero  $(d, l)$  se calcula una sola vez.

Por ende ti tenemos 10 posibles digitos y  $n$  posibles longitudes

Entonces el total de estados es  $10 * n$

cada estado sabemos que toma  $O(k)$  para hacer la suma de vecinos .

Por ende

$$O(10 * n * k)$$

Como 10 y  $k$  son constantes entonces

$$O(n)$$

- Usando su programa, encuentre las combinaciones totales posibles para  $n = 10$ .

```
El numero de combinaciones para n=10 el resultado es: 1806282
```

Figure 2: alt text

## 2.2 Problema 2 [40 puntos]

Dada una matriz cuadrada de 0's y 1's, calcule el tamaño de la cruz (símbolo de suma +) más grande formada por 1's.

Input :

```
grid = [
[ 1 , 0 , 1 , 1 , 1 , 1 , 0 , 1 , 1 , 1 ] ,
[ 1 , 0 , 1 , 0 , 1 , 1 , 1 , 0 , 1 , 1 ] ,
[ 1 , 1 , 1 , 0 , 1 , 1 , 0 , 1 , 0 , 1 ] ,
[ 0 , 0 , 0 , 0 , 1 , 0 , 0 , 1 , 0 , 0 ] ,
[ 1 , 1 , 1 , 0 , 1 , 1 , 1 , 1 , 1 , 1 ] ,
[ 1 , 1 , 1 , 1 , 1 , 1 , 1 , 1 , 1 , 0 ] ,
[ 1 , 0 , 0 , 0 , 1 , 0 , 0 , 1 , 0 , 1 ] ,
[ 1 , 0 , 1 , 1 , 1 , 1 , 0 , 0 , 1 , 1 ] ,
[ 1 , 1 , 0 , 0 , 1 , 0 , 1 , 0 , 0 , 1 ] ,
[ 1 , 0 , 1 , 1 , 1 , 1 , 0 , 1 , 0 , 0 ]
]
```

Output : 17 Explicación: La cruz ( símbolo de suma +) mas grande de 1s se encuentra abajo, teniendo un tamaño de 17.

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 0 | 1 | 0 |   | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 |   | 1 | 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 0 |   | 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 |   | 1 | 1 | 1 | 1 | 1 |
| — | — | — | — |   | — | — | — | — | 0 |
| 1 | 0 | 0 | 0 |   | 0 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 |   | 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |   | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |   | 1 | 0 | 1 | 0 | 0 |

Figure 3: alt text

Input :

```
grid = [
[ 1 , 1 , 1 , 1 , 1 , 1 ] ,
[ 1 , 0 , 1 , 1 , 0 , 1 ] ,
[ 0 , 1 , 1 , 0 , 0 , 1 ] ,
[ 1 , 1 , 1 , 1 , 1 , 1 ] ,
[ 1 , 0 , 0 , 1 , 0 , 1 ] ,
[ 1 , 0 , 1 , 1 , 0 , 0 ]
]
```

Output : 0 Explicación: No se puede construir una cruz (+) más grande usando los 1s.

- Coloque un enlace a un GitHub Gist privado con la solución. Recuerde que debe crear también un video en donde muestre la ejecución de su código. Use como prueba el input proporcionado. (30 puntos).

Link del Código

Link al Video

- Encuentre el tiempo de complejidad para este algoritmo. Recuerde, deje su procedimiento. (10 puntos)

“{algoritmo} n = len(matriz)

left = [[0]\*n for \_ in range(n)] right = [[0]\*n for \_ in range(n)] top = [[0]\*n for \_ in range(n)] bottom = [[0]\*n for \_ in range(n)]

for i in range(n): for j in range(n): if matriz[i][j] == 1: # T(1) left[i][j] = (left[i][j-1] if j > 0 else 0) + 1 top[i][j] = (top[i-1][j] if i > 0 else 0) + 1

## Llenar right y bottom

for i in range(n-1, -1, -1): for j in range(n-1, -1, -1): if matriz[i][j] == 1: right[i][j] = (right[i][j+1] if j < n-1 else 0) + 1 bottom[i][j] = (bottom[i+1][j] if i < n-1 else 0) + 1

## Encontrar la cruz más grande

max\_cross = 0 for i in range(n): for j in range(n): if matriz[i][j] == 1: size = min(left[i][j], right[i][j], top[i][j], bottom[i][j]) if size >=2: max\_cross = max(max\_cross, 4 \* (size - 1) + 1) # cada brazo más el centro

return max\_cross “

Para su análisis vamos a verificar cada parte del código

Para llenar las matrices

$$T(n) = \sum_{i=0}^n \sum_{j=0}^n 4T(1) + C$$

Porque son 4 matrices al final nos da

$$T(n) = \sum_{i=0}^n n + C$$

$$T(n) = n * n$$

Lo que da

$$T(n) = n^2$$

Para el resto del codigo tendríamos que

$$T(n) = \sum_{i=0}^n \sum_{j=0}^n 3T(1) + C \sum_{i=0}^n \sum_{j=0}^n 3T(1) + C \sum_{i=0}^n \sum_{j=0}^n 4T(1) + C$$

Al simplificarlo nos daría

$$T(n) = 3 \sum_{i=0}^n \sum_{j=0}^n 3T(1) + 3T(n) + 4T(n) + C$$

$$T(n) = 9T(1)(n+1)^2 + 3T(n) + 4T(n) + C$$

$$T(n) = 9T(1)(n+1)^2 + 7T(n) + C$$

$$T(n) - 7T(n) = 9T(1)(n+1)^2 + C$$

$$-6T(n) = 9T(1)(n+1)^2 + C$$

$$T(n) = -\frac{9T(1)(n+1)^2}{6} - \frac{C}{6}$$

$$T(n) = -\frac{3T(1)(n+1)^2}{2} - \frac{C}{6}$$

Ahora expandimos  $(n+1)^2$

$$T(n) = -\frac{3T(1)(n^2 + 2n + 2)}{2} - \frac{C}{6}$$

$$T(n) = -\frac{3T(1)n^2}{2} - 3T(1)n - \frac{3T(1)}{2} - \frac{C}{6}$$

Tomamos el resto como constantes.

$$T(n) = -\frac{3T(1)n^2}{2}$$

Sabemos que  $3/2$  es constante entonces

$$T(n) = n^2$$

Por lo tanto el valor total de su complejidad seria

$$T(n) = n^2 + n^2 = 2n^2$$

Lo que da al final que la complejidad del algoritmo es de

$$T(n) = O(n^2)$$

### 2.3 Problema 3 [5 puntos]

**Planteamiento** Verdadero o Falso: Una solución óptima para un knapsack problem siempre contendrá el objeto  $i$  con la mayor relación valor-costo  $v_i/c_i$ . Explique

**Solución** Falso, pues no necesariamente aquí lo que se busca es que se maximice lo que podemos llevar no el objeto íesimo que tiene la mayor cantidad.

También porque el objeto con mayor costo puede ser demasiado grande, espacio que objetos más pequeños podrían aportar mucho más.

### 2.5 Problema 5 [15 puntos]

**Planteamiento** Dada una matriz  $T$  de  $m \times n$  sobre un campo (como los reales), demuestre que  $(S, I)$  es una matroide, en donde  $S$  es el conjunto de las columnas de  $T$  y  $A$  pertenece a  $I$  si y solo si las columnas en  $A$  son linealmente independientes



**Solucion** Verificamos los 3 axiomas de la matroide

- 1 El conjunto vacío es independiente

El conjunto vacío es linealmente independiente por lo que no pertenece a  $I$

- 2. Todo subconjunto de un conjunto independiente es independiente

Para este como  $A$  pertenece a  $I$  un conjunto independiente y  $B$  es un subconjunto de  $A$ .

Las columnas de  $A$  perteneciente a  $I$  son linealmente independientes,  $B \subseteq A$ , esto quiere decir que las columnas de  $B$  son parte de  $A$ . Pero si esto existe quiere decir que  $A$  es dependiente de  $B$  y viceversa y esto no es posible así que  $B \in I$

- 3 Propiedad de intercambio

Sabemos que  $|A| < |B|$  se debe encontrar  $e \in B - A$  tal que  $A \cup \{e\} \in I$ .

Si  $s$  es  $s = |A| < |B| = f$  entonces  $A$  contiene  $s$  columnas linealmente independientes, mientras que  $B$  contiene  $f$  columnas linealmente independientes.

Si todas las columnas de  $B$  estuvieran en  $A$  o en el espacio  $V$ , entonces tendríamos  $\text{span}(B) \subseteq V$ , y la dimensión de  $\text{span}(B)$  sería a lo sumo  $f$

Pero esto es contradictorio pues  $B$  tiene  $f > s$  columnas linealmente independientes, lo que implica que  $\dim(\text{span}(B)) = f > s$ .

Por lo tanto en  $B$  debe haber una columna que no sea de  $A$ . Por ende tenemos que  $\{e\} \cup A$  son linealmente independientes

Porque se cumplen estas 3 axiomas podemos decir que  $(S, I)$  es una matroide